



Wir betrachten ein soziales Netzwerk im Stil von Aufgabe 3 aus Übungsblatt 7 und modellieren es als gerichteten Graphen. Dabei entspricht jeder Knoten einer Person und eine Kante von Knoten A zu Knoten B sagt aus, dass die Person A der Person B folgt.

Im Folgenden sei ein solcher Graph als Adjazenzliste gegeben, wobei statt Personennamen der Einfachheit halber natürliche Zahlen verwendet werden. Wir implementieren diese Adjazenzliste in Python als Dictionary (Hash-Tabelle) `graphOut`, wobei die Schlüssel die Knoten des Graphen und die Werte Sets (Mengen) von Knoten sind, sodass folgender Zusammenhang gilt: für jeden Knoten  $i$  ist `graphOut[i]` die Menge aller Knoten  $j$ , sodass der Graph die Kante  $(i, j)$  enthält.

```
graphOut = {0: {17}, 1: {50, 36, 7, 38, 71, 22}, 2: {16, 19, 12, 29, 47}, 3: {33, 18, 4
```

a) Speichere den Graphen in einer Datenstruktur, in der nicht die adjazenten Knoten der ausgehenden Kanten, sondern die adjazenten Knoten der eingehenden Kanten schnell gefunden werden können.

*Hinweis: schreibe ein Python-Skript, das die oben gegebene Datenstruktur `graphOut` in eine Datenstruktur `graphIn` überführt, die dieselbe Form hat, aber folgenden Zusammenhang implementiert: für jeden Knoten  $i$  ist `graphIn[i]` die Menge aller Knoten  $j$ , sodass der Graph die Kante  $(j, i)$  enthält. Welche Laufzeit hat dein Skript?*

b) Finde die beliebteste Person im Netzwerk.

*Hinweis: eine Person ist beliebt, wenn ihr besonders viele folgen.*

c) Wir nehmen an, dass zwei Personen befreundet sind, wenn sie sich gegenseitig folgen. Erstelle einen Graphen, der die Freundschaftsverhältnisse aus dem oben gegebenen Graphen wiedergibt.

*Hinweis: speichere auch den neuen Graphen als Adjazenzliste. In welchem Verhältnis steht diese zu `graphOut`?*

d) Finde die zwei Personen, die die meisten gemeinsamen Freunde haben, aber selbst nicht miteinander befreundet sind.